GOOGLE COLAB / EXISTING SCRIPT TESTING

Google Colab for karaoke file processing

Testing existing MP3 and MP4 processing scripts, choosing a payment option and running resumable batches.

Recommended use: test the scripts your development team is already preparing, then run controlled batches once compatibility, output quality and recovery have been demonstrated. Save finished files back to persistent storage for use by the karaoke portal.

What Colab provides

Colab is a browser-based notebook environment with remote computing resources, including GPU access. The existing scripts perform the conversion; Colab supplies the environment in which they run. [1] Your particular scripts have not been inspected or tested for this guide.

Is a paid subscription required?

No. Colab has a free option, and paid access can be purchased with or without a subscription. [2, 3]

Option	Payment model	How to consider it for this project
Free Colab	No paid subscription required. Resource limits and availability apply. [2]	Use first to check installation and a small representative processing run.
Pay As You Go	Purchase compute units without a subscription. [3]	Consider a measured paid batch if the free allocation limits useful testing.
Colab Pro / Pro+	Subscriptions with recurring compute units and additional benefits. [1]	Compare for repeated processing after measuring actual usage. Paid access is still limited.
Eligible Google AI plan	Some paid Google AI plans include Colab benefits. Eligibility applies. [4]	Check existing account benefits before buying an additional plan.

Purchase decision: use the actual price and benefits displayed for your account on Google's pricing page [5]. A subscription price alone does not establish the cost of converting the catalogue. Measure compute consumption, transfers, storage needs and quality-review effort.

01 / TECHNICAL VALIDATION

Check the existing scripts

Test the processing stages the scripts actually require. A file that already has suitable audio, lyrics or video should follow the appropriate existing workflow; do not assume that every file needs AI separation or new lyric timing.

Check	What the developer should demonstrate
Environment and dependencies	The required language version, libraries, models, fonts, assets and command-line tools install reproducibly. Replace machine-specific paths or desktop dialogs where necessary.
Actual GPU use	Confirm which stages use the GPU and measure them. Selecting a GPU runtime does not automatically accelerate the code or all video operations. [2]
File access and capacity	A representative batch can be read, staged and saved reliably. Allow space for intermediate files as well as final outputs; permanent storage capacity must be checked separately.
Restart and recovery	Completed outputs and a persistent progress record survive session loss. Restarting continues from saved state, with failed items clearly identified.
Output acceptance	Check audio/video quality, duration, lyric offsets, any embedded lyrics and playback in the actual karaoke player. A successful script exit is only one part of acceptance.
Complete throughput	Measure setup/model downloads, input transfer, processing, rendering, retries and output saving. Track compute-unit consumption separately from elapsed time and human review.

Suggested pilot: 10-20 representative files

1. **Select a useful sample.** Include MP3 and MP4 files, different durations and each processing path used by the existing scripts. This is an initial pilot, not proof of catalogue-wide compatibility.
2. **Run the existing workflow.** Use a recorded script version and configuration. Preserve the originals and compare the results with expected outputs.
3. **Validate in the player.** Listen and watch, confirm lyric synchronisation and check that the portal/player accepts the resulting files.
4. **Interrupt and resume.** End the session deliberately, start a new one and verify that saved work is recovered without unnecessary reprocessing.
5. **Record the evidence.** Report successful, failed and review-needed files; time by stage; output size; compute consumption; and the reason for each failure.

02 / BATCH OPERATION

Make progress survive each session

Colab runtimes are temporary: virtual machines can be deleted after inactivity and have a maximum lifetime. GPU availability and usage limits vary, including on paid plans. Keep originals, completed outputs and progress records in persistent storage. [2]

Recommended batch sequence

1. **Read the saved job list.** Identify pending files and their required processing stages. Use stable file IDs or checksums to distinguish inputs.
2. **Stage a manageable batch.** Copy only the files needed for the next work period into temporary working space.
3. **Run the existing scripts.** Record the script/model version and settings, and capture errors against the relevant input.
4. **Validate and save the result.** Check the required outputs, save them to permanent storage and confirm the save before marking the file complete.
5. **Update the progress record.** Store completion status, output location, timing and any quality-review flag. A new session should continue from this record.

What the development team should provide

For a concrete compatibility review: the processing scripts, their dependency/build files, the required input/output formats and a few representative files. No new conversion workflow is assumed by this guide.

Before scaling up, provide a reproducible Colab setup, a working interruption/recovery test, output-quality results and an estimate based on measured throughput and compute usage. Include transfer time, storage and manual review in the estimate.

> **Practical next step:** complete the small pilot, then use its results to choose free access, a Pay As You Go batch or a subscription. Keep the processing setup reproducible and each completed file safely saved.

Official references and purchasing link

Technical documentation checked on 9 November 2026. Plan prices and account benefits should be confirmed at checkout. The test and batch procedures above are recommendations for your team.

[1] Google for Developers - Colab overview
https://developers.google.com/colab

[2] Google Colab FAQ - runtimes, resource limits and GPU use
https://research.google.com/colaboratory/faq.html

[3] Google Colab - Making the Most of your Colab Subscription
https://colab.research.google.com/notebooks/pro.ipynb

[4] Google Developers Blog - Colab benefits in Google AI plans
https://developers.googleblog.com/colab-is-now-part-of-your-google-ai-plan/

[5] Google Colab - current paid options and account pricing
https://colab.research.google.com/signup